Towards Privacy-Enhanced and Robust Clustered Federated Learning

Yang Xu, Yunlin Tan, Cheng Zhang, Peng Sun, Yibang Zhang, Ju Ren, *Senior Member, IEEE*, Hongbo Jiang, *Senior Member, IEEE*, and Yaoxue Zhang, *Senior Member, IEEE*

***Abstract*—Clustered federated learning (CFL) leverages data distribution similarities to cluster clients, facilitating personalized model training under data heterogeneity. However, most existing CFL schemes pose potential privacy risks for clients (e.g., gradient inversion attacks) as they rely on individual gradients for clustering. This also renders them incompatible with secure aggregation mechanisms that are widely employed in federated learning for privacy protection. Moreover, CFL introduces the risk of malicious clients dominating several clusters and conducting poisoning attacks therein, thereby threatening secure model training. To address these issues, we propose ProCFL, a Privacy-Enhanced and Robust CFL framework incorporating gradient-free clustering and peer validation. Specifically, we first design a new protocol for measuring data distribution similarity among clients without using their gradient information. Then, we transform the client clustering process into a weighted set covering problem and introduce a diversity-optimized clustering algorithm to achieve near-optimal clustering results while eliminating any need for prior knowledge. Furthermore, we develop a post-hoc detection mechanism that employs peer validation to identify and discard malicious client models. Extensive experimental evaluation of ProCFL validates its superior model robustness and accuracy performance compared to existing schemes.**

***Index Terms*—Clustered federated learning, gradient-free clustering, intersection similarity metric, malicious cluster attack.**

Received 22 May 2024; revised 19 February 2025; accepted 26 February 2025. Date of publication 3 March 2025; date of current version 3 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62472163, Grant 62272154, and Grant 62472158, in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3125 and Grant 2024RC3102, in part by the Natural Science Foundation of Hunan Province under Grant 2024JJ5096 and Grant 2023JJ40174, in part by the Science & Technology Talents Lifting Project of Hunan Province under Grant 2023TJ-N23, in part by the Training Program for Excellent Young Innovators of Changsha under Grant kq2209008, in part by the Hunan Provincial Department of Education Excellent Youth Project under Grant 24B0039, in part by the Young Elite Scientists Sponsorship Program by CAST under Grant 2023QNRC001, in part by the Open Topics from the Lion Rock Labs of Cyberspace Security under Grant LRL24015, and in part by the Hunan Provincial Innovation Foundation for Postgraduate under Grant CX20230389. Recommended for acceptance by E. Ngai. *(Corresponding author: Peng Sun.)*

Yang Xu, Yunlin Tan, Cheng Zhang, Peng Sun, and Yibang Zhang are with the College of Cyber Science and Technology, Hunan University, Changsha 410082, China (e-mail: xuyangcs@hnu.edu.cn; tanyunlin@hnu.edu.cn; zhangchengcs@hnu.edu.cn; psun@hnu.edu.cn; zhangyb@hnu.edu.cn).

Ju Ren and Yaoxue Zhang are with the Department of Computer Science and Technology, BNRist, Tsinghua University, Beijing 100084, China (e-mail: renju@tsinghua.edu.cn; zhangyx@tsinghua.edu.cn).

Hongbo Jiang is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: hongbojiang@hnu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3547149

I. Introduction

FEDERATED learning (FL) has garnered significant interest and has found numerous practical applications due to its ability to collaboratively build machine learning models with multiple clients while preserving the privacy of their data [1], [2], [3]. However, conventional FL paradigms that train a single shared global model for all clients suffer from significant performance degradation when there is non-independent and identically distributed (non-IID) data among clients [4], [5].

To address this issue, researchers have proposed numerous personalized FL schemes such as clustered federated learning (CFL) [6], [7], [8], federated multitask learning [9], and federated meta learning [10]. These methods enable the customization of a distinct model for each client based on their specific data distribution. Among these, CFL has gained particular popularity due to its compatibility with existing FL frameworks and ease of deployment [11], [12]. CFL achieves model personalization by leveraging the similarity in clients' data distribution to classify them into different clusters before training and aggregating models within their respective clusters, leading to improved performance in non-IID scenarios.

Most existing CFL schemes (e.g., [13], [14]) rely on precise individual gradients to cluster clients with similar local data distributions. This reliance makes them vulnerable to advanced privacy attacks (e.g., gradient inversion attacks [15]) that can accurately reconstruct clients' private training data (such as pixel-level images) from their shared gradients. More importantly, existing privacy-preserving techniques, such as differential privacy and secure aggregation [16], [17], [18], are not effective in addressing privacy threats during client clustering. This is because these methods only provide the server with aggregated or perturbed gradients, preventing access to precise individual client gradients, which are essential for effective client clustering. While certain CFL schemes may not rely on individual gradients, they may still necessitate access to inaccessible or other sensitive prior information, such as the number of clusters or clients' social relationships [7], [8], [19], [20].

Moreover, like conventional FL, CFL is also susceptible to poisoning attacks launched by malicious clients [21], which has rarely been explicitly accounted for by current CFL solutions. Even worse, the clustering operation in CFL introduces a new security threat. Specifically, a few (say less than 50% of the total clients) malicious clients may collude to gather and dominate several clusters and then conduct poisoning attacks to ruin the

aggregated cluster models (which is referred to as malicious cluster attacks). Thus far, many poisoning attack detection and defense methods have been proposed to protect the model security in FL [22], [23], [24], [25], [26]. However, these methods generally require detecting uploaded local models in each training round, resulting in significant computational overhead. Moreover, these methods are inadequate in effectively identifying and defending against poisoning attacks when malicious clients constitute the majority within clusters. Several other schemes [27], [28], [29] enhance the model security in CFL by directly excluding the cluster models though only a few clients' malicious models are aggregated in these clusters, thus overlooking benign client models and leading to a biased global model. Therefore, how to efficiently and effectively detect and defend against malicious cluster attacks in CFL remains unresolved.

To address the above challenges, in this work, we propose a Privacy-Enhanced and Robust CFL framework named ProCFL that incorporates gradient-free clustering and peer validation to mitigate privacy risks of clients and resist poisoning attacks in CFL. Specifically, we first design a new protocol to measure data distribution similarity free of gradients. This protocol employs the private set intersection (PSI) technique, a cryptographic protocol that enables two parties to compute the intersection of their sets without revealing any additional information about the sets' contents. By using PSI, we can evaluate the similarity between sets related to data distributions of clients, thereby enhancing client privacy.

Then, ProCFL further considers leveraging data diversity to achieve client clustering without relying on prior knowledge. Specifically, we transform the client clustering process into a weighted set covering problem (WSCP). This is an NP-hard problem that attempts to find subsets from a set of subsets (each with a cost) that cover all involved elements at minimum cost. After a series of equivalent transformations, such as viewing these subsets as client clusters, we design a greedy diversity-optimized clustering algorithm. In this way, one can group clients with diverse data classes into multiple clusters, corresponding to using their data in multiple clusters, promising enhanced model performance. Notably, our clustering method is fully compatible with existing privacy-preserving mechanisms for CFL. By integrating ProCFL with secure aggregation techniques, it enables a gradient-free approach to CFL.

Finally, in cases where the CFL system encounters malicious client poisoning attacks, our design prioritizes system robustness over strict privacy to ensure cluster models remain functional even under a malicious majority. Specifically, we build a post-hoc detection mechanism where clients with similar data distributions in other clusters perform peer validation on small samples to identify and exclude malicious model updates.

The major contributions of this paper are as follows:

- To our knowledge, we propose the first Privacy-Enhanced and Robust CFL framework, namely ProCFL, which can achieve personalized model training in non-IID and adversarial scenarios without relying on gradients or any prior knowledge when clustering clients.
- We develop a new protocol adopting the PSI technique to measure client data distribution similarity, offering desirable privacy protection to clients. Based on this protocol, we transform the client clustering process into a WSCP and design a diversity-optimized clustering algorithm to obtain near-optimal clustering results.
- We build a lightweight post-hoc attack detection mechanism using peer validation to effectively identify and resist malicious cluster attacks.
- We conduct an extensive experimental evaluation of ProCFL. The results validate that our scheme *enhances client privacy during clustering and resistance to malicious cluster attacks during model training and achieves better model performance than existing schemes*.

The rest of this paper is organized as follows. Section II reviews related work. Section III presents the problem statement. Section IV elaborates on the design of ProCFL. Section V provides a security analysis. Section VI presents the experimental results, followed by some discussions in Section VII. Finally, Section VIII concludes the paper.

II. Related Work

In this section, we review and discuss related work from two aspects, i.e., clustered federated learning and poisoning attack detection and defenses.

A. Clustered Federated Learning

Traditional FL frameworks undergo significant model performance deterioration in non-IID data scenarios as a single global model cannot satisfy different clients' demands for model accuracy. CFL has been proposed and widely adopted to circumvent the non-IID issue, which can customize satisfactory models for each client according to its data distribution.

HypCluster [13] is the initial work dividing clients into different clusters, which requires predefined cluster structures to determine which clusters clients are assigned to. Then, Sattler et al. [6] formally presented the concept of CFL, which refers to recursively separating clients into groups based on the cosine similarity of gradients. Ghosh et al. [7] proposed the Iterative Federated Clustering Algorithm (IFCA), where clients select the optimal model parameters and determine their clustering identities in each round. These iterative CFL schemes require frequent clustering operations during model training, leading to additional system overheads. Researchers in [14] identified that efficient CFL could be achieved by only one-shot clustering based on clients' latest computed gradients when the global model stabilizes. In line with [14], many other one-shot clustering algorithms have been proposed for CFL [30], [31], [32], promising desirable model training performance.

Nonetheless, most of the above schemes rely on clients' gradients for clustering, which results in potential privacy threats induced by advanced privacy attacks (e.g., [33], [34], [35]) or require prior knowledge that is inaccessible in practice. Existing CFL methods typically rely on individual client gradients to perform clustering, which renders commonly used privacy preservation mechanisms incompatible with these schemes. As a result, these mechanisms fail to provide adequate privacy protection for CFL. For instance, differential privacy techniques

introduce noise to client gradients, which disrupts accurate client clustering and ultimately degrades global model performance. Similarly, homomorphic encryption transforms plaintext gradients into encrypted gradients, making it impossible for a server without decryption keys to conduct clustering. Furthermore, when secure aggregation is employed, the server only has access to the aggregated gradients of all clients, which also prevents effective client clustering.Unlike the methods mentioned above, our gradient-free client clustering protocol eliminates the reliance on gradients for clustering. This design ensures compatibility with existing privacy-preserving mechanisms, thereby enhancing privacy protection throughout the entire CFL model training process. Second, during the clustering process, we incorporate data diversity [19], [36] to divide clients with diverse data classes into multiple clusters. These two aspects substantially increase the effectiveness and practicability of our proposed CFL scheme.

B. Poisoning Attack Detection and Defenses

Adversaries may conduct poisoning attacks against FL to manipulate the model training process and ruin the final model performance. Accordingly, many poisoning attack detection and defense methods have been proposed to enhance model security.

For example, Krum [37] detects and filters malicious local models by only accepting the one with the smallest Euclidean distance to others. Zhao et al. [38] proposed a sampling-based detection mechanism to identify Byzantine adversaries' submitted models. DeepSA [22] offers a defense mechanism by pre-training a DNN-based detector. Li et al. [23] filters out malicious models by calculating the cosine similarity among all local model updates. Zhao et al. [24] adopted the client-side peer validation technique to verify malicious models.

Nevertheless, the aforementioned methods necessitate the evaluation of each uploaded local model (i.e., computing statistical distances or testing model losses) in every round, which introduces a significant computational burden. Furthermore, these schemes generally work assuming that benign clients constitute the majority. However, this assumption may not hold in CFL scenarios, as malicious clients can collude to gain control over specific clusters. Several poisoning attack detection schemes have recently been proposed for CFL, which exclude outlier models by designing various robust clustering algorithms [27], [28], [29]. However, in these algorithms, once a cluster is flagged as a malicious cluster, all client models therein will be removed directly or less weighed in model aggregation, leading to biased model training. In contrast, in this work, we build a lightweight post-hoc attack detection mechanism to discover malicious client models fine-grainedly.

III. System Model and Problem Statement

In this section, we first provide a high-level system overview. Then, we present the threat model. Finally, we describe the design goals and challenges of ProCFL. We summarize the notations used in this paper in Table I.

TABLE I
Summary of Notations

Symbol	Description
$\mathcal{N}$	Set of clients
$\mathcal{D}_n$	The local dataset of client n
$\mathcal{K}$	Set of clusters
CL_k	The cluster leader in cluster k
T	The total number of model training rounds
λ	The proportion of global learning rounds
$\boldsymbol{w}^t$	Global model in round t
$\boldsymbol{w}_n^{t+1}$	The updated local model of client n in round t
$\boldsymbol{w}_k^{t+1}$	The aggregated cluster model of cluster k in round t
η	Learning rate for local model update
$g(\boldsymbol{w}; b)$	The computed stochastic gradient on model $\boldsymbol{w}$
$\mathcal{B}_n$	Set of mini-batches splitted from $\mathcal{D}_n$
E_n	Epoch number for local model updating at client n
$\mathcal{C}_n$	Set of clusters into which client n is categorized
$f(\boldsymbol{w}_k^t; \mathcal{D}_n)$	Testing loss of the cluster model $\boldsymbol{w}_k^t$
n_k	The number of clients in cluster k
n_k^m	The number of malicious clients in cluster k
f	The total number of malicious clients
$\boldsymbol{M}_{sim}$	Data distribution similarities matrix
$\mathcal{L}$	Set of possible labels of the considered dataset
$\mathcal{Q}_n$	Set of pairs of sample counts and labels
$\mathcal{X}_n$	The data distribution-related set of client n
ISM_n	ISM vector of client n
$c(\mathcal{S}_n)$	The cost of cluster $\mathcal{S}_n$
$\mathcal{C}_M$	Malicious cluster
$\mathcal{V}$	Validation committee

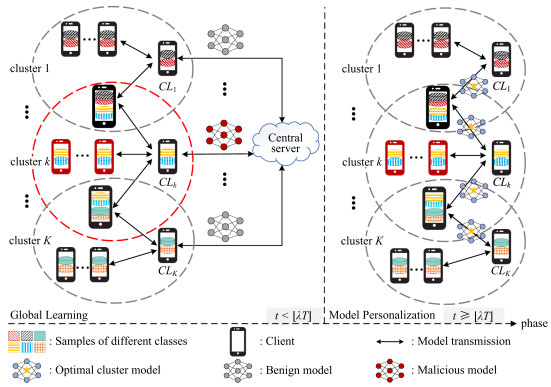


Fig. 1. System overview of ProCFL.

A. System Setting

As shown in Fig. 1, similar to prior studies [30], we consider a CFL system consisting of the following three entities.

- *Clients:* There is a set $\mathcal{N} = \{1, \ldots, N\}$ of clients in the CFL system. Each client $n \in \mathcal{N}$ owns a private data set $\mathcal{D}_n$, which statistically differs from others. All clients are clustered into K groups based on their data distribution. Clients within the same cluster collaboratively train a personalized model that fits their data distribution.
- *Cluster Leaders:* Assume that N clients are organized into a set $\mathcal{K} = \{1, \ldots, K\}$ of clusters, each of which contains n_k clients. Each cluster $k \in \mathcal{K}$ randomly selects an internal client to serve as the cluster leader denoted as CL_k, which takes charge of aggregating cluster models of all clients within cluster k.

- *Central Server:* The server initializes the global model and sets hyperparameters. During training, the server is responsible for clustering clients into k clusters and aggregating the global model.

Workflow: All clients first jointly train the global model, upon which clients within each cluster then iteratively train personalized models. Specifically, in the initialization phase, the server initializes the model parameters and hyperparameters and sets the total number of rounds of model training T, the number of rounds of global learning $\lfloor \lambda T \rfloor$ ($\lambda \in [0,1]$), and the number of rounds of personalized training $\lfloor (1-\lambda)T \rfloor$. The server also utilizes a clustering algorithm to cluster clients. In each round of ProCFL, the following steps are sequentially performed (see Algorithm 1):

- *Local Update at Clients:*
  - In each global learning round t ($t < \lfloor \lambda T \rfloor$), each client $n \in \mathcal{N}$ first splits its local dataset $\mathcal{D}_n$ into mini-batches $\mathcal{B}_n$ of size B_n. Then, client n will perform $E_n \frac{D_n}{B_n}$ steps of local model update starting from the global model $\boldsymbol{w}^t$ using mini-batch stochastic gradient descent, where E_n is the local epochs at client n. If we use $g(\boldsymbol{w}; b)$ to represent the mini-batch stochastic gradient computed based on the model $\boldsymbol{w}$ and a mini-batch $b \in \mathcal{B}_n$, then one step of local model update is expressed as
$$\boldsymbol{w}^{t,i} = \boldsymbol{w}^{t,i-1} - \eta g\left(\boldsymbol{w}^{t,i-1}; b\right), \tag{1}$$
where $i = 1, \ldots, E_n \frac{D_n}{B_n}$, $\boldsymbol{w}^{t,0} = \boldsymbol{w}^t$, $\boldsymbol{w}^{t, E_n \frac{D_n}{B_n}} = \boldsymbol{w}_n^{t+1}$, and η is the local learning rate.
  - In each model personalization round t ($t \geq \lfloor \lambda T \rfloor$), each client $n \in \mathcal{N}$ conducts the same local training except that $\boldsymbol{w}^{t,0} = \boldsymbol{w}_k^t$. In other words, the starting point is the corresponding cluster model.
    Clients with diverse classes of data samples that are grouped into multiple clusters will receive multiple cluster models from different cluster leaders. Among these cluster models, the clients will select the one with the minimum testing loss on their local data for the local model update. Formally, the cluster index k is determined as
$$k = \arg\min_{k \in \mathcal{C}_n} f\left(\boldsymbol{w}_k^t; \mathcal{D}_n\right), \tag{2}$$
where $\mathcal{C}_n$ denotes the set of clusters that client n is simultaneously grouped into, and $f(\boldsymbol{w}_k^t; \mathcal{D}_n)$ is the testing loss of the cluster model $\boldsymbol{w}_k^t$ on client n's local dataset $\mathcal{D}_n$.
- *Cluster Aggregation at Cluster Leaders:*
  After accomplishing local training, each client uploads the updated local model to its corresponding cluster leaders. Once having received the local models from all clients in cluster k, each cluster leader $CL_k (\forall k \in \mathcal{K})$ performs cluster aggregation via
$$\boldsymbol{w}_k^{t+1} = \frac{1}{n_k} \sum_{n=1}^{n_k} \boldsymbol{w}_n^{t+1}, \tag{3}$$
where n_k denotes the number of clients in cluster k.

Algorithm 1: Model Training Process of ProCFL.

Input: $\boldsymbol{w}^0$: initialized global model; K: the number of clusters; T: model training rounds; λ: the proportion of global learning rounds; E_n: the number of local epochs for each client $n \in \mathcal{N}$;
Output: Cluster models $\boldsymbol{w}_1^T, \ldots, \boldsymbol{w}_K^T$ and the global model $\boldsymbol{w}^{\lfloor \lambda T \rfloor}$;

for each round $t = 0, 1, \ldots, \lfloor \lambda T \rfloor - 1$ **do**
  Each client $n \in \mathcal{N}$ processes:
  $\mathcal{B}_n \leftarrow$ (splits $\mathcal{D}_n$ into mini-batches of size B_n)
  for each local epoch e from 1 to E_n **do**
    for each batch $b \in \mathcal{B}_n$ **do**
      $\boldsymbol{w}^{t,i} = \boldsymbol{w}^{t,i-1} - \eta g(\boldsymbol{w}^{t,i-1}; b)$
    end for
  end for // Local Model Update
  Send $\boldsymbol{w}_n^{t+1}$ to the corresponding cluster leaders
  Each cluster leader CL_k ($\forall k \in \mathcal{K}$) processes:
  $\boldsymbol{w}_k^{t+1} \leftarrow \frac{1}{n_k} \sum_{n=1}^{n_k} \boldsymbol{w}_n^{t+1}$ // Cluster Aggregation
  Send $\boldsymbol{w}_k^{t+1}$ to the server
  Server processes:
  $\boldsymbol{w}^{t+1} \leftarrow \frac{1}{K} \sum_{k=1}^{K} \boldsymbol{w}_k^{t+1}$ // Global Aggregation
  Send $\boldsymbol{w}^{t+1}$ to clients via cluster leaders
end for
for each round $t = \lfloor \lambda T \rfloor, \lfloor \lambda T \rfloor + 1, \ldots, T-1$ **do**
  Repeat lines $2-11$ except starting from the selected cluster model $\boldsymbol{w}_k^t$ according to (2)
  Send $\boldsymbol{w}_k^{t+1}$ to corresponding clients
end for
return Cluster models $\boldsymbol{w}_1^T, \ldots, \boldsymbol{w}_K^T$ and the global model $\boldsymbol{w}^{\lfloor \lambda T \rfloor}$;

  Next, CL_k either sends $\boldsymbol{w}_k^{t+1}$ back to the clients in the cluster for the next round of local training (model personalization phase) or uploads $\boldsymbol{w}_k^{t+1}$ to the server for global model aggregation (global learning phase).
- *Global Aggregation at the Central Server:*
  During the global learning phase, the server will receive the cluster models $\boldsymbol{w}_k^{t+1}$ ($k \in \mathcal{K}$) in each round t and then aggregate them to obtain a new global model, i.e.,
$$\boldsymbol{w}^{t+1} = \frac{1}{K} \sum_{k=1}^{K} \boldsymbol{w}_k^{t+1}. \tag{4}$$

B. Threat Model

Attacker: We consider two types of attacks: (1) *data reconstruction attacks* by an honest-but-curious server, and (2) *poisoning attacks* by malicious clients (either injected by an adversary or compromised benign clients).

A1: Data Reconstruction Attacks occur when an honest-but-curious server attempts to reconstruct clients' private local training data, such as pixel-level images, from shared gradients during both client clustering and model training processes. These attacks have been extensively studied in the literature (e.g., [15]).

A2: Poisoning Attacks involve malicious clients sending crafted, poisoned local model updates to the cluster leader or server for aggregation. These updates may result from tampering with local training data or falsifying model parameters, aiming to disrupt the CFL model training process. In line with previous studies on poisoning attacks [21], [39], this work focuses on untargeted poisoning attacks, where the goal is to degrade the learned cluster and global models, leading to high error rates for testing examples indiscriminately. Besides, following prior studies [27], [28], [29], we consider that the number of malicious clients f is less than half the total clients (i.e., $f < N/2$). We specifically consider the following two attack forms:

- *Dispersed Attack:* Malicious clients can attack CFL independently (i.e., without collusion). That is, these malicious clients are dispersed across multiple clusters. In this case, malicious clients probably cannot dominate their located clusters. That is, benign clients are in the majority of these clusters. However, the associated cluster models can also be poisoned.
- *Malicious Cluster Attack:* Malicious clients may collude in advance to build highly similar data distributions. In this way, these malicious clients can be grouped into the same cluster and dominate it (for example, when cluster k is dominated, we have $n_k^m/n_k > 50\%$, where n_k^m denotes the number of malicious clients in cluster k), forming malicious cluster attacks.

Defender: To mitigate privacy threats during model training, clients can utilize existing privacy protection mechanisms, such as differential privacy and secure aggregation, to prevent the server or cluster leader from accessing their raw individual gradients. However, as noted earlier, these privacy-preserving techniques are ineffective in addressing privacy risks during client clustering. This limitation arises because current CFL methods rely on precise individual gradients for clustering. Therefore, we are motivated to design a gradient-free client clustering protocol.

For poisoning attacks, we follow previous research (e.g., [40], [41]) and assume that the server has access to a small number of test samples, which can be collected from public datasets or manually labeled. We further assume that the cluster leader CL_k ($k \in \mathcal{K}$) is honest and faithfully performs cluster model aggregation. Such an assumption is reasonable because the server can easily detect hostile behavior (e.g., tampering with the cluster model) by the cluster leader. For example, the server can periodically ask clients in the cluster to upload their local model updates and aggregate them to obtain a baseline cluster model. The server can then determine whether the cluster leader is compromised by comparing its submitted cluster model to the benchmark.

C. Design Goals and Challenges

The design goals of ProCFL can be summarized as follows:

Privacy Enhancement: ProCFL should prevent data reconstruction attacks during client clustering, ensuring that clients' private training data cannot be reconstructed.

Robustness: ProCFL must effectively resist poisoning attacks under non-IID data distributions, including both dispersed attacks and malicious cluster attacks.

Effectiveness: ProCFL should deliver comparable or better model performance than traditional FL and gradient-based CFL, without relying on any prior knowledge.

To achieve the above design goals, we need to address the following challenges.

- How can we implement gradient-free client clustering to address privacy threats at the root level, considering the limitations of existing privacy-preservation mechanisms in client clustering?
- How can we effectively exclude malicious clients in non-IID scenarios, particularly when malicious clients are the majority within a cluster?

IV. Design of ProCFL

A. Overview

In this work, we propose ProCFL, a novel CFL framework that not only achieves privacy-enhanced client clustering by implementing gradient-free clustering but also effectively resists malicious attacks. ProCFL incorporates three fundamental techniques, i.e., data distribution similarity measurement, diversity-optimized client clustering, and malicious cluster attack detection, as shown in Fig. 2.

Specifically, to achieve privacy-enhanced client clustering, we first present a gradient-free metric named intersection similarity metric (ISM) to measure the similarity of data distribution among clients. ISM is an asymmetric metric between clients, which ensures that clients' private data distribution information is well preserved when calculating ISM. Furthermore, we design an ISM computing protocol utilizing the PSI technique to obtain the similarity of data distribution among clients without compromising their privacy. Subsequently, to boost CFL's performance, we propose dividing clients with diverse classes of data samples into multiple clusters. Then, we transform the client clustering process into a WSCP and design a greedy diversity-optimized clustering algorithm to solve it. Finally, we propose a post-hoc malicious cluster attack detection mechanism to identify and resist malicious cluster attacks, guaranteeing the model security.

Now, we describe the workflow of ProCFL as follows:

1) Each client $n \in \mathcal{N}$ obtains an ISM vector based on the data distribution similarity measurement protocol (which will be elaborated in Section IV-B), which records its data distribution similarity with the remaining clients;
2) The server collects the ISM vectors from all clients and constructs a similarity matrix denoted as $\boldsymbol{M}_{sim}$, which records the data distribution similarities among clients;
3) Based on $\boldsymbol{M}_{sim}$, the server runs the diversity-optimized clustering algorithm (which will be introduced in Section IV-C) to divide clients into different clusters;
4) The model training process of ProCFL starts under the orchestration of clients, cluster leaders, and the server. The training process is terminated when the cluster models converge to a preset accuracy level or the pre-determined number of training rounds T is reached;

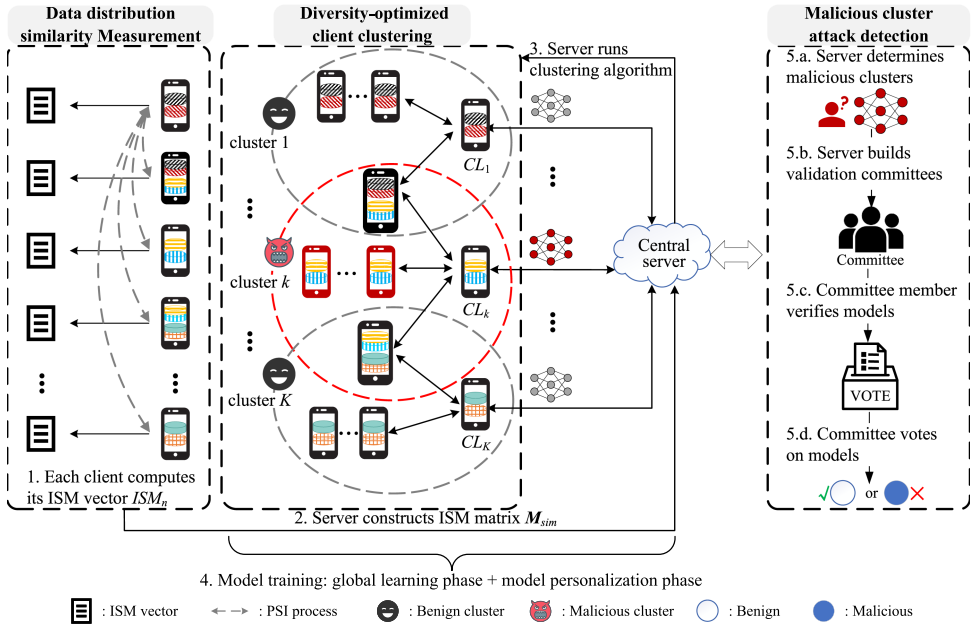


Fig. 2. The schematic diagram of ProCFL.

5) When a degradation in the accuracy of the global model is witnessed, the server initiates the attack detection procedures (which will be detailed in Section IV-D):
   a) The server first determines the malicious clusters by testing the received cluster models on its collected public dataset;
   b) After determining the malicious clusters, to accurately distinguish which clients within the identified malicious clusters are malicious, the server selects several clients who have similar data distributions as clients in the malicious clusters to build a validation committee;
   c) Then, each committee member verifies each client model within the identified malicious clusters;
   d) Finally, all committee members vote to determine whether a client model is benign or malicious. According to the voting results, the server can exclude malicious client models and accept benign ones.

B. Gradient-Free Distribution Similarity Measurement

To support client clustering in CFL, we first need to characterize the similarity of data distribution among clients. We first present a gradient-free data distribution similarity metric to eliminate the need to acquire clients' gradients or other prior knowledge when implementing clustering. Then, we design a privacy-preserving metric computation protocol to determine the metric values among clients (i.e., data distribution similarity degrees).

An appropriate data distribution similarity metric is fundamental for correctly clustering clients. Existing metrics are usually based on clients' raw gradients, which introduce large computation overheads for complex deep learning models. More seriously, using the original gradient directly as a metric introduces serious privacy risks (e.g., gradient inversion attacks). In addition, the metrics should not disclose clients' data distribution information either, which may expose their sensitive information like personal preferences. Hence, those data distribution similarity metrics with symmetric properties (e.g., Jaccard similarity coefficient [42]) are undesirable. If a symmetric metric is used to compute the similarity, then a particular client is able to determine whether the other party has a similar data distribution to itself based on this similarity.

In light of the above considerations, we design ISM. To formally characterize ISM, we first construct a data distribution-related set for each client. Specifically, we let $\mathcal{L} = \{1, \ldots, L\}$ represent the set of possible labels of the considered dataset. Then, each client $n \in \mathcal{N}$ counts the number of data samples X_n^l belonging to each class $l \in \mathcal{L}$. We can summarize the set of possible labels and the corresponding number of data samples belonging to each label in a set $\mathcal{Q}_n = \{(1, X_n^1), \ldots, (l, X_n^l), \ldots, (L, X_n^L)\}$. For each pair of (l, X_n^l), we construct a set according to

$$\mathcal{Q}_n^l = \bigcup_{i=1}^{X_n^l} (l \times Q_{max} + i)\,. \tag{5}$$

(5) is used to assign a unique, non-overlapping index to each data sample in a client's local training dataset for constructing distribution-related sets and performing subsequent ISM calculations. Importantly, an exact value for Q_{max} is not required; it only needs to be larger than the maximum number of data samples across all labels. Then, the data distribution-related set for each client n is constructed as

$$\mathcal{X}_n = \bigcup_{l=1}^{L} \mathcal{Q}_n^l. \tag{6}$$

Then we formally characterize ISM in Definition 1.

Definition 1. (Intersection Similarity Metric): For each client $n \in \mathcal{N}$, its ISM w.r.t. client m is computed as

$$ISM_n[m] = \frac{|\mathcal{X}_n \cap \mathcal{X}_m|}{|\mathcal{X}_n|}. \tag{7}$$

Here, $ISM_n[m] \in [0, 1]$ captures the proportion of the number of common elements shared between $\mathcal{X}_n$ and $\mathcal{X}_m$ over the cardinality of $\mathcal{X}_n$. Thus, $ISM_n[m]$ can indicate the similarity level of data distribution between clients n and m. Note that ISM is an asymmetric metric since $ISM_n[m] = 1$ only reflects that the two associated clients have similar but not necessarily identical data distribution.

As the data distribution-related set $\mathcal{X}_n$ may contain sensitive information about client n, such as personal preferences, it is essential to avoid directly exposing $\mathcal{X}_n$ to other clients when calculating the ISM metric. Therefore, we propose a privacy-preserving method for computing the ISM metric using PSI techniques [43]. PSI is a cryptographic protocol that allows two parties to compute the intersection of their sets without revealing any information about their sets' contents. In this work, we employ a Rivest–Shamir–Adleman (RSA)-based PSI protocol (RSA-PSI) developed in [44] to calculate the ISM in a secure and private manner.[1] Specifically, given $\mathcal{X}_n$, each client n obtains the intersections with others, i.e., $\mathcal{X}_n \cap \mathcal{X}_1, \ldots, \mathcal{X}_n \cap \mathcal{X}_N$. Then, we can group the ISM values of client n w.r.t. other clients (including itself) into an ISM vector, i.e.,

$$\begin{aligned} ISM_n &= \left(\frac{|\mathcal{X}_n \cap \mathcal{X}_1|}{|\mathcal{X}_n|}, \frac{|\mathcal{X}_n \cap \mathcal{X}_2|}{|\mathcal{X}_n|}, \ldots, \frac{|\mathcal{X}_n \cap \mathcal{X}_N|}{|\mathcal{X}_n|} \right) \\ &= \left(ISM_n[1], ISM_n[2], \ldots, ISM_n[N] \right). \end{aligned} \tag{8}$$

We need to emphasize that ISM does not expose the client's privacy. First, although the server can obtain the similarity between two clients through ISM, this similarity does not contain any statistical information. In addition, the server cannot obtain the raw data used by the clients for training through ISM either. Second, a client can obtain the similarity to other clients. However, due to the symmetric feature of ISM, client n can not determine whether the data distribution is similar to client m even if $ISM_n[m] = 1$.

C. Diversity-Optimized Client Clustering

To implement client clustering, the server collects the ISM vector ISM_n from each client $n \in \mathcal{N}$ and constructs an ISM matrix $\boldsymbol{M}_{sim} \in \mathbb{R}^{N \times N}$, i.e.,

$$\boldsymbol{M}_{sim} = \begin{bmatrix} ISM_1[1] & \cdots & ISM_1[N] \\ \vdots & \ddots & \vdots \\ ISM_N[1] & \cdots & ISM_N[N] \end{bmatrix}. \tag{9}$$

Each row of $\boldsymbol{M}_{sim}$ corresponds to the ISM vector of a client.

[1]Note that our work concentrates on creating a gradient-free data distribution similarity metric (i.e., ISM). We utilize existing PSI techniques to compute the metric while preserving privacy. Therefore, in addition to the RSA-PSI scheme [44] utilized in this study, more efficient PSI techniques can also be employed to calculate the ISM.

After obtaining the ISM matrix $\boldsymbol{M}_{sim}$, we can proceed with client clustering. In this work, we do not directly apply traditional clustering or matching algorithms for two main reasons. First, traditional clustering techniques typically assign clients to a single cluster, which limits clients with diverse data classes from contributing to the training of multiple cluster models. This restricts their ability to reduce the divergence among cluster models, ultimately impairing global model performance. Second, while some soft clustering methods [19], [36] aim to address the above issue, they generally assume symmetric similarity between clients. However, the data distribution similarity captured by our ISM metric is asymmetric, making these methods inapplicable.

To address these issues, we propose a diversity-optimized client clustering method. Specifically, we transform the client clustering process into a WSCP based on the ISM matrix Msim. We then design efficient algorithms to solve this problem and perform client clustering. In what follows, we first provide the background on WSCP, then explain how we reformulate the client clustering process as a WSCP, and finally describe our solution.

1) Weighted Set Covering Problem Introduction: A WSCP [45] is a mathematical optimization problem that involves selecting a minimum number of sets, each with an associated cost or weight, such that the union of the selected sets covers a given ground set.

Formally, a WSCP is defined as follows. Let Ω be a set of m elements, i.e., $\Omega = \{1, \ldots, m\}$, and let $\mathcal{S}$ be a collection of p subsets of Ω, i.e., $\mathcal{S} = \{\mathcal{S}_1, \ldots, \mathcal{S}_p\}$ with each element being a subset of Ω. Each subset in $\mathcal{S}$ has an associated cost or weight $c_1, \ldots, c_p$. Then, the goal is to select a minimum-cost subcollection of $\mathcal{S}$ that covers all elements of Ω. That is, we aim to find a subset $\mathcal{S}'$ of $\mathcal{S}$ such that

- The union of all sets in $\mathcal{S}'$ contains every element of Ω.
- The aggregate weights of sets in $\mathcal{S}'$ is minimized.

Mathematically, if we use $x_i (i = 1, \ldots, p)$ to indicate whether $\mathcal{S}_i$ in $\mathcal{S}$ is included in $\mathcal{S}'$ ($x_i = 1$ implies being included, and $x_i = 0$ otherwise), then the WSCP can be formulated as

$$\text{minimize} \quad \sum_{i=1}^{p} c_i x_i \tag{10a}$$

$$\text{subject to} \quad \sum_{i: j \in \mathcal{S}_i} x_i \geq 1, \ \forall j = 1, \ldots, m \tag{10b}$$

$$x_i \in \{0, 1\}, \quad \forall i = 1, \ldots, p \tag{10c}$$

- Objective function (10a) is to minimize the total weights.
- Constraint (10b) enforces that each element in Ω should appear at least once in the sets in $\mathcal{S}'$.

Notably, the WSCP has an implication that each element in Ω may exist in multiple sets of $\mathcal{S}'$, which inspires us to design the diversity-optimized clustering algorithm next.

2) Clustering Problem Transformation: To transform the client clustering process into a WSCP, we first consider the ground set Ω as a set of candidate clients. Moreover, each element in $\mathcal{S}$ can be regarded as a potential cluster of clients (i.e., a subset of Ω). To enhance model training performance, we

allow clients with diverse classes of data samples to be divided into multiple clusters simultaneously (i.e., diversity-optimized client clustering). This corresponds to the fact that each element in Ω may exist in multiple sets of $\mathcal{S}$. Then, the client clustering process is equivalent to finding a subset $\mathcal{S}'$ of $\mathcal{S}$ with the minimum cost. Nevertheless, to accomplish the client clustering problem transformation, we still need to address the following three issues:

- The candidate set $\mathcal{S}$ is unknown *a priori* as the number and structure of clusters are unavailable.
- Each cluster should be unique, given that identical clusters yield redundant cluster models.
- The associated cost of selecting a cluster $\mathcal{S}_i$ from $\mathcal{S}$ is unknown.

To address the above issues, we first construct $\mathcal{S}$, i.e., obtaining all possible clusters based on a pre-determined data distribution similarity threshold α (measured by ISM). Specifically, for each client $n \in \mathcal{N}$, those clients with ISM values higher than α will be grouped (along with client n) into a cluster $\mathcal{S}_n$, thus becoming a potential element in $\mathcal{S}$. Note that in this way, clients with diverse classes of data will be added to multiple clusters due to their large data distribution similarity with multiple clients (lines 1–8). Then, to avoid redundant clusters in $\mathcal{S}$, we apply the de-duplication operation to $\mathcal{S}$ (line 9).

Next, we need to properly characterize the cost of each cluster in $\mathcal{S}$ (lines 10–13). Considering that the objective of client clustering is to group clients with similar data distributions into clusters, the cost of each cluster should capture its overall data distribution similarity accordingly. Specifically, the cost of each cluster $\mathcal{S}_n$ in $\mathcal{S}$ is computed as

$$c(\mathcal{S}_n) = \frac{1}{\sum_{j \in \mathcal{S}_n} ISM_n[j]}. \tag{11}$$

Here, the cost function (11) indicates that a cluster with a large overall data distribution similarity has a small cost.

Thus far, we have successfully addressed the aforementioned three issues and transformed the client clustering problem with diversity optimization into a WSCP. With the cost function (11), the solution to the WSCP corresponds to a set of clusters that covers all clients and presents the largest intra-cluster data distribution similarity overall. Next, we show how to obtain the solution.

3) Client Clustering Problem Solution: The WSCP is NP-hard, and no known efficient algorithm can solve it for large instances. Alternatively, we design a greedy solution. We first define the payload of each candidate cluster $\mathcal{S}_n$ in $\mathcal{S}$ in Definition 2.

Definition 2. (Payload): For any candidate clusters $\mathcal{S}_n$ in $\mathcal{S}$, if we use $\mathcal{I}$ to represent the already included clients in the selected clusters, its payload is computed as

$$Payload(\mathcal{S}_n) = \frac{c(\mathcal{S}_n)}{|\mathcal{S}_n \setminus \mathcal{I}|}, \tag{12}$$

which captures the average cost of the clients in $\mathcal{S}_n$ that have not been included in the selected clusters.

Our basic idea for solving the WSCP is to greedily select (*without replacement*) the candidate cluster with the minimum payload in $\mathcal{S}$ one by one and add the associated clients to $\mathcal{I}$ until $\mathcal{I}$ covers all clients (lines 14–21). We summarize our diversity-optimized client clustering algorithm in Algorithm 2.

Algorithm 2: Diversity-Optimized Client Clustering.

Input: $\boldsymbol{M}_{sim} \in \mathbb{R}^{N \times N}$: similarity matrix; α: similarity threshold; $\Omega = \{1, \ldots, N\}$: the clients to be clustered; $\mathcal{S}' = \emptyset$: initial empty cluster set;
Output: Near-optimal $\mathcal{S}'$

1: Initialize $\mathcal{S}$: $\forall \mathcal{S}_n \in \mathcal{S}, \mathcal{S}_n = \emptyset$, where $n \in \mathcal{N}$;
2: **for** $n = 1, \ldots, N$ **do**
3: **for** $j = 1, \ldots, N$ **do**
4: **if** $\boldsymbol{M}_{sim}[n][j] \geq \alpha$ **then**
5: $\mathcal{S}_n.add(j)$;
6: **end if**
7: **end for**
8: **end for** // Get all potential clusters $\mathcal{S}_n$
9: Remove duplicate elements from $\mathcal{S}$;
10: Initialize $c(\mathcal{S}_n) = 0$ for each $\mathcal{S}_n$ in $\mathcal{S}$;
11: **for** $\forall \mathcal{S}_n \in \mathcal{S}$ **do**
12: Calculate the $c(\mathcal{S}_n)$ according to (11);
13: **end for** // Get the cost of $\mathcal{S}_n$
14: Initialize $\mathcal{I} = \emptyset$: $\mathcal{I}$ represents the included clients in $\mathcal{S}'$;
15: **while** $\mathcal{I} \neq \Omega$ **do**
16: Find $\mathcal{S}_n \in \mathcal{S}$ with the minimal payload (computed based on (12));
17: $\mathcal{I} = \mathcal{I} \cup \mathcal{S}_n$;
18: $\mathcal{S}'.add(\mathcal{S}_n)$;
19: $\mathcal{S} = \mathcal{S} \setminus \mathcal{S}_n$;
20: **end while**
21: **return** $\mathcal{S}'$; // Get the near-optimal clustering result

D. Post-Hoc Attack Detection

A straightforward defense strategy for *dispersed attacks* is to perform malicious model detection before each round of cluster model aggregation, which introduces large additional overheads to model training, especially in large-scale CFL systems. For *malicious cluster attacks*, it is highly nontrivial for existing detection schemes (which work under the assumption that benign clients are in the majority) to identify malicious client models within the malicious cluster. Alternatively, we can directly discard malicious cluster models (once identified). However, such a rough operation will erase the contribution of some benign clients to model training, leading to a biased model in non-IID scenarios.

To address the above issues, we propose a post-hoc malicious cluster attack detection mechanism to detect malicious client models after identifying malicious cluster attacks and dispersed attacks. In practice, the server continuously monitors the global model accuracy. When a decreased accuracy is witnessed, it will test the received cluster models based on a small public dataset. Then, the post-hoc detection mechanism is activated and executed as follows:

Algorithm 3: Validation Committee Generation.

Input: $\boldsymbol{M}_{sim} \in \mathbb{R}^{N\times N}$: similarity matrix; $\mathcal{C}_M = \{1, \ldots, M\}$: malicious cluster; $\mathcal{V} = \emptyset$: initial empty validation committee set; d: the size of the validation committee;
Output: $\mathcal{V}$

1: **if** $d < |\mathcal{C}_M| < N - |\mathcal{C}_M|$ **then**
2: **while** $|\mathcal{V}| \neq |\mathcal{C}_M|$ **do**
3: **for** each client $m \in \mathcal{C}_M$ **do**
4: Find j such that $\boldsymbol{M}_{sim}[m][j]$ is maximal;
5: **if** $j \notin \mathcal{C}_M$ and client $j \notin \mathcal{V}$ **then**
6: $\mathcal{V}.add$(client j); // Process next client m
7: **else**
8: $\boldsymbol{M}_{sim}[m][j] = -1$;
9: **Go to** line 4; // Let j not be selected again
10: **end if**
11: **end for**
12: **end while**
13: **else**
14: Add all remaining clients not in $\mathcal{C}_M$ to $\mathcal{V}$;
15: **end if**
16: **for** each $v_i \in \mathcal{V}$ **do**
17: Using (13) to obtain the overall similarity Sim_{v_i};
18: **end for**
19: Select d candidates with the largest Sim_{v_i} from $\mathcal{V}$ as the final $\mathcal{V}$;
20: **return** $\mathcal{V}$; // Get the validation committee $\mathcal{V}$

- *Step 1:* For each malicious cluster $\mathcal{C}_M = \{1, \ldots, M\}$, the server selects d clients that are not in this malicious cluster and are most similar to the clients in this malicious cluster to form a validation committee. We summarize this process in Algorithm 3. The detail process of Step 1 is described as follows:
  - When the number of clients in $\mathcal{C}_M$ is less than 50% of total clients (i.e., $M < N/2$) and larger than the predefined size of the validation committee, for each client m in $\mathcal{C}_M$, we select the one outside $\mathcal{C}_M$ with the largest ISM value w.r.t. client m as the candidate validation committee member. Otherwise, we directly select all $N - M$ clients outside $\mathcal{C}_M$ as the candidate validation committee members (lines 1–15).
  - We calculate the overall data distribution similarity of all clients in $\mathcal{C}_M$ w.r.t. each candidate validation committee member v_i (lines 16–18). Formally, for each v_i, we have
$$Sim_{v_i} = \sum_{m=1}^{M} \boldsymbol{M}_{sim}[m][v_i]. \tag{13}$$
  - For more reliable detection result, we select d clients with the largest Sim_{v_i} from these candidates to form the final committee (lines 19–20).
- *Step 2:* After establishing a validation committee for each detected malicious cluster, the clients within these clusters are required to send their local models to the corresponding validation committee for evaluation. Since committee members may infer client privacy from model parameters, we allow clients in malicious clusters to protect their privacy using Gaussian noise before sharing parameters.
- *Step 3:* Each committee member uses its local data to test the model received from the clients in the malicious cluster. The committee members then individually determine whether a model is malicious or benign according to
$$H_{verify} = \begin{cases} \text{malicious}, & if\ acc_{model} \leq acc_{mean} \\ \text{benign}, & if\ acc_{model} > acc_{mean} \end{cases}, \tag{14}$$
where acc_{model} represents the test accuracy of the model evaluated by the committee member, and acc_{mean} is the average test accuracy of all models examined by the same committee member.
- *Step 4:* After completing the validation, the validation committee members vote on the clients in the malicious cluster. The clients considered malicious by more than half of the committee members will be discarded. The remaining clients deemed benign are aggregated into a new cluster to join the subsequent federated learning process.

Regarding network bandwidth, direct transmission of model parameters from clients under suspicion to validation committee members can be avoided. Instead, when the server identifies a malicious cluster, it instructs the cluster head to transmit all local models from clients in the cluster to the server. The server then forwards these local models to validation committee members as part of the process of distributing the newly updated global model to clients. This approach minimizes uplink bandwidth usage while requiring only a modest increase in downlink bandwidth, which remains acceptable.

V. Security Analysis

In this section, we perform a security analysis to highlight the effectiveness of our post-hoc malicious cluster attack detection method in safeguarding model security from malicious cluster attacks and accurately identifying poisoned client models within such clusters. The formal security analysis is presented in Theorem 1.

Theorem 1: The probability of malicious clients' emergence in the validation committee designated for a detected malicious cluster is comparably insignificant. Therefore, detecting whether a client model in the malicious cluster is malicious or not is reliable as most committee members can perform such detection.

Proof: Let p represent the overall proportion of malicious clients existing in the CFL system. Suppose that an identified malicious cluster with a size of n_k has n_k^m malicious clients, and $n_k^m/n_k > 50\%$. In our post-hoc detection mechanism, d clients will be selected as the validation committee from the remaining $N - n_k$ clients. Then, the probability P_e that each client model in the malicious cluster will be sent to a validation committee with e malicious members (given that the overall committee size

is d) is expressed as

$$P_e = \sum_{n_k^m=\lfloor \frac{n_k}{2} \rfloor+1}^{n_k} \frac{\binom{N\cdot(1-p)}{n_k-n_k^m}\binom{N\cdot p}{n_k^m}}{\binom{N}{n_k}} \times \frac{\binom{N\cdot p-n_k^m}{e}\binom{N\cdot(1-p)-n_k+n_k^m}{d-e}}{\binom{N-n_k}{d}}. \quad (15)$$

The probability P_e is determined by two factors. The first term of P_e indicates the probability of a malicious cluster attack, which requires that $n_k^m/n_k > 50\%$. The second term describes the probability of forming a validation committee with e malicious clients out of d committee members. For instance, let us consider a CFL system with $N = 100$ clients, where 50% of the clients are malicious. If a malicious cluster model with $n_k = 10$ is detected, a validation committee of size $d = 10$ clients is constructed to examine the client models in the malicious cluster. Using (15), we can compute P_e for $e = 6, 7, 8, 9, 10$, yielding $P_e \approx 0.07, 0.04, 0.01, 0.002, 0.0001$, respectively. Note that we take $e = 6, 7, 8, 9, 10$ as examples as these correspond to cases where most committee members will make unreliable verification. Thus, the final detection results on models are unreliable. However, the sufficiently small values of P_e indicate that it is highly unlikely for the validation committee to make unreliable malicious client model detection. If the percentage of malicious clients is only 10%, the probability becomes $P_e \approx 0$ for any $e > d/2$.

Moreover, it can be inferred from (15) that P_e approaches zero when $N \cdot p - n_k^m < e$. This suggests that a malicious cluster with a sufficiently large number of malicious clients is highly unlikely to assign client models to a validation committee with e malicious members. Thus, the detection results of client models can be considered reliable. □

We will show in Section VI how the proportion of malicious clients and the number of committee members affect the detection performance.

VI. Experimental Evaluation

In this section, we first present the experimental setting, and then evaluate ProCFL against different attacks. The experiments are conducted on a computer with a 64-core AMD EPYC 7742 CPU processor and 192 GB of RAM.

A. Experimental Setup

1) Datasets: We conduct experiments on image datasets MNIST [46], Fashion-MNIST [47], CIFAR-10 [48], and TinyImage [49], and the text dataset Texas-100 [50]. We follow prior works [14], [39], [51] to divide the training and testing sets, and consider the following non-IID settings:

- *Quantity-based label imbalance:* In this setting, each client only has data samples of h different labels, and a smaller h indicates a higher non-IID degree.
- *Distribution-based label imbalance:* In this setting, each client is assigned a proportion of samples for each label based on the Dirichlet distribution. Specifically, we sample $p_{h,n} \sim Dir(\beta)$ and allocate a $p_{h,n}$ proportion of instances with label h to client n. Here, the symbol $Dir(\cdot)$ denotes the Dirichlet distribution, and β represents the non-IID degree, wherein a smaller value of β corresponds to a higher non-IID degree.
- *Features distribution imbalance:* In this setting, each client has the same number of labels and samples, but the images in the training set will be horizontally flipped with a probability of 0.5 and then rotated 15 degrees.

2) Models: We adopt the following models in our experiments.

- For CIFAR-10, we train a convolutional neural network (CNN) [39] ($Conv2d(3*6*5) \rightarrow ReLU \rightarrow MaxPool2d(2*2) \rightarrow Conv2d(6*16*3) \rightarrow ReLU \rightarrow MaxPool2d(2*2) \rightarrow ReLU \rightarrow Linear(400*120) \rightarrow ReLU \rightarrow Linear(120*84) \rightarrow ReLU \rightarrow Linear(84*10)$).
- For MNIST, we train a CNN [51]($Conv2d(1*16*3) \rightarrow ReLU \rightarrow MaxPool2d(2*2) \rightarrow Conv2d(16*32*3) \rightarrow ReLU \rightarrow MaxPool2d(2*2) \rightarrow Linear(1568*128) \rightarrow ReLU \rightarrow Linear(128*10)$).
- For FMNIST, we train a CNN [14]($Conv2d(1*16*5) \rightarrow BatchNorm2d(16) \rightarrow ReLU \rightarrow MaxPool2d(2*2) \rightarrow Conv2d(16*32*3) \rightarrow BatchNorm2d(16) \rightarrow ReLu \rightarrow MaxPool2d(2*2) \rightarrow Linear(1568*10)$).
- For Texas-100, we train a 5-layer full connected neural network [52]($Linear(6169*1024) \rightarrow Tanh \rightarrow Linear(1024*512) \rightarrow Tanh \rightarrow Linear(512*256) \rightarrow Tanh \rightarrow Linear(256*128) \rightarrow Tanh \rightarrow Linear(128*100)$).
- For TinyImage, we train a Resnet18 [53], and the last full connected layer is $Linear(512*200)$.

3) Baseline Methods: We compare ProCFL with the following methods:

- FedAvg [1]. A conventional FL algorithm (without client clustering) is employed here as a benchmark.
- CFL+Cos [6]. It is an iterative CFL scheme that exploits the cosine similarity between the clients' gradients to group the client population into clusters.
- IFCA [7]. It is an iterative CFL scheme, which determines the number of clusters in advance and allows clients to select the optimal cluster model for themselves to achieve clustering.
- FL+HC [14]. It is a one-shot CFL scheme, which implements client clustering through a hierarchical clustering algorithm on the server side.

4) CFL Settings and Parameter Configurations: By default, we consider a CFL system with one central server. For MNIST, Fashion-MNIST and CIFAR-10, the number of clients is 100. For Texas-100 and TinyImage, the number are 50 and 20 respectively. By default, the total number of model training rounds in ProCFL is $T = 50$. Following prior studies [6], [14], we choose the first 10 rounds for global learning, while the remaining for model personalization, i.e., $\lambda = 0.2$. For local model training,

TABLE II
PARAMETER CONFIGURATIONS FOR CFL BASELINES UNDER DIFFERENT DATASETS

Datasets	Baselines		
	IFCA	FL+HC	CFL+Cos
MNIST	$K = 3$	$\gamma = 3$	$\varepsilon_1 = 0.30$ $\varepsilon_2 = 0.90$
FMNIST	$K = 3$	$\gamma = 10$	$\varepsilon_1 = 0.10$ $\varepsilon_2 = 1.00$
CIFAR-10	$K = 3$	$\gamma = 2$	$\varepsilon_1 = 0.05$ $\varepsilon_2 = 0.20$
Texas-100	$K = 15$	$\gamma = 3$	$\varepsilon_1 = 0.90$ $\varepsilon_2 = 3.60$
TinyImage	$K = 25$	$\gamma = 3$	$\varepsilon_1 = 2.00$ $\varepsilon_2 = 7.50$

[1] K is the pre-determined number of clusters in IFCA.
[2] γ is the distance threshold in FL+HC.
[3] ε_1 and ε_2 are the splitting parameters in CFL+Cos.

TABLE III
IMPACT OF DATA DISTRIBUTION SIMILARITY THRESHOLD ON CLUSTERING AND MODEL TRAINING UNDER MNIST, FMNIST, AND CIFAR-10

α	MNIST		FMNIST		CIFAR-10	
	$h = 2$	β=0.5	$h = 2$	β=0.5	$h = 2$	β=0.5
0.45	0.9313	0.9461 ↑	0.8840	0.8781 ↑	0.3830	0.3406 ↑
0.50	0.9911	0.9444 ↓	0.9848	0.8773 ↓	0.7732	0.3502 ↑
0.55	0.9911	0.9486 ↑	0.9848	0.8806 ↑	0.7732	0.3619 ↑
0.60	0.9911	**0.9497** ★	0.9848	0.8873 ↑	0.7732	0.3917 ↑
0.65	0.9911	0.9475 ↓	0.9848	0.8809 ↓	0.7732	0.4036 ↑
0.70	0.9911	0.9454 ↓	0.9848	0.8849 ↑	0.7732	0.4272 ↑
0.75	0.9911	0.9456 ↑	0.9848	0.8894 ↑	0.7732	0.4264 ↓
0.80	0.9911	0.9432 ↓	0.9848	0.8896 ↑	0.7732	0.4533 ↑
0.85	0.9911	0.9350 ↑	0.9848	**0.8960** ★	0.7732	0.4693 ↑
0.90	0.9911	0.9315 ↓	0.9848	0.8902 ↓	0.7732	**0.4902** ★

★ indicates the highest accuracy under different threshold values α.

we use the SGD optimizer and set the local epoch $E_n = 1$, learning rate $\eta = 0.01$, and batch size $B_n = 256$ for each client $n \in \mathcal{N}$ for MNIST, FMNIST and CIFAR-10. For Texas-100, we use the Adam optimizer and set Learning rate $\eta = 0.001$, and batch size $B_n = 64$. For TinyImage, We use the Adam optimizer and set learning rate $\eta = 0.001$, and batch size $B_n = 256$. The parameter configurations for the baselines are summarized in Table II.

B. Experimental Results

1) Clustering Performance: We should determine proper data distribution similarity threshold values α to obtain desirable clustering and model training performance. Thus, we first investigate how α affects the clustering process and the final cluster model accuracy. We simulate two different non-IID settings, i.e., quantity-based label imbalance and distribution-based label imbalance. We summarize the average cluster model accuracy after $T = 50$ rounds of model training with α varying from 0.45 to 0.9 with a step size of 0.05in Table III and 0.15 to 0.5 with a step size of 0.05in Table IV. We observe that for the non-IID setting of $\beta = 0.5$, the model accuracy roughly increases first and then decreases with α. Moreover, the model accuracy remains stable for the non-IID setting of h when α reaches a certain value. Given these experimental results, we choose $\alpha = 0.60$, $\alpha = 0.85$, $\alpha = 0.90$, $\alpha = 0.30$ and $\alpha = 0.30$ for datasets MNIST, FMNIST, CIFAR-10, Texas-100, and TinyImage, respectively, to achieve the best possible clustering and model training performance.

TABLE IV
IMPACT OF DATA DISTRIBUTION SIMILARITY THRESHOLD ON CLUSTERING AND MODEL TRAINING UNDER TEXAS-100 AND TINYIMAGE

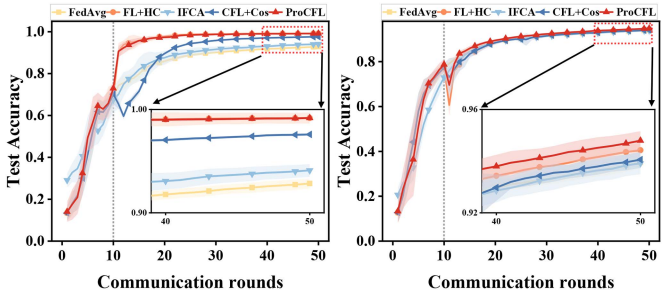
α	Texas-100		TinyImage	
	$h = 10$	β=0.5	$h = 20$	β=0.5
0.15	0.5673	0.6151 ↑	0.2248	0.2387 ★
0.20	0.6092	0.6203 ★	0.3335	0.2303 ↓
0.25	0.6416	0.6088 ↓	0.3441	0.2339 ↑
0.30	0.7611	0.6127 ↑	0.3653	0.2301 ↓
0.35	0.7616	0.6119 ↓	0.3655	0.2287 ↓
0.40	0.7822	0.5930 ↓	0.3652	0.1617 ↓
0.45	0.7651	0.4813 ↓	0.3710	0.1522 ↓
0.50	0.7875	0.4574 ↓	0.3637	0.1496 ↓

★ indicates the highest accuracy under different threshold values α.

2) Necessity of Gradient-Free Clustering in ProCFL: Before investigating the performance of ProCFL, we conduct experiments to illustrate that differential privacy (DP) is unsuitable for addressing privacy issues in client clustering. Specifically, we compare the model test accuracies obtained by common CFL methods (i.e., IFCA and CFL+Cos) under CFL w/wo DP (Note that DP is achieved by adding a random Gaussian noise with a variance of 0.01 to the clients' gradients). Fig. 8 demonstrates that applying DP during client clustering results in significant accuracy loss. This highlights that DP is not suitable for addressing privacy issues during client clustering in CFL, thus necessitating our gradient-free clustering design.

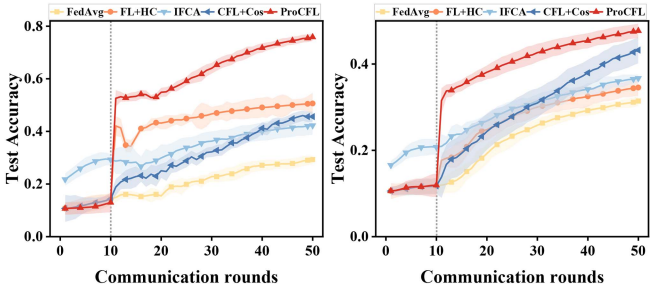
3) Comparison With Baselines: We evaluate ProCFL against baseline methods by comparing the average test accuracy of client models after T training rounds on five datasets. Results are averaged over 10 independent runs, except for TinyImage, which uses 5 runs. The mean values are shown with solid lines, while the shaded regions indicate standard deviations.

As shown in Fig. 3 to 7, our method consistently achieves the highest accuracy under two different non-IID settings compared to the baselines. This can be attributed to two reasons: First, our method directly clusters based on clients' datasets, resulting in more accurate clustering outcomes compared to clustering methods based on local models, which in turn leads to higher cluster model accuracy. Second, our method leverages the diversity in clients' data distributions, further enhancing model accuracy in non-IID settings. Figs. 6 and 7 plot the average test accuracy of multiple experimental runs, where the shaded region represents the three-sigma range. We can see that performance improvements of our approach are particularly significant on complex datasets such as TinyImage and Texas-100.



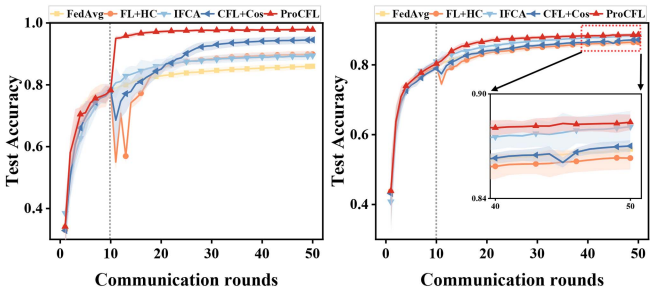
(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 3. Comparison of testing accuracy on MNIST.



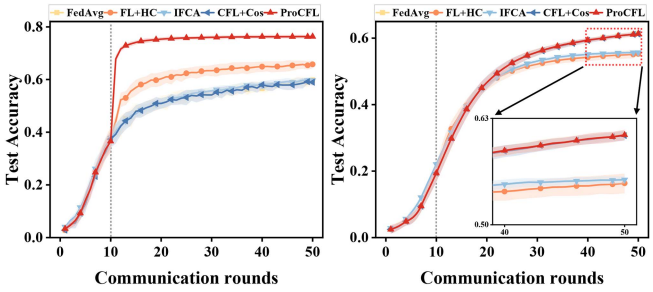
(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 4. Comparison of testing accuracy on CIFAR-10.



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

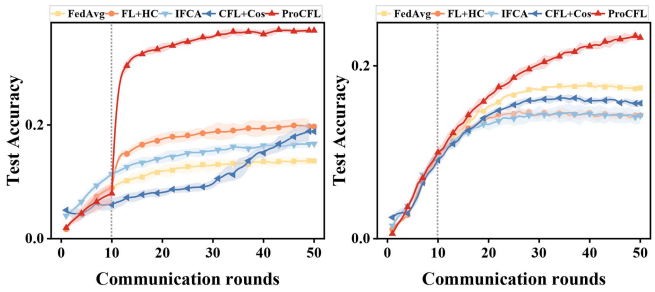
Fig. 5. Comparison of testing accuracy on FMNIST.



(a) The non-IID setting of $h = 10$. (b) The non-IID setting of $\beta = 0.5$.

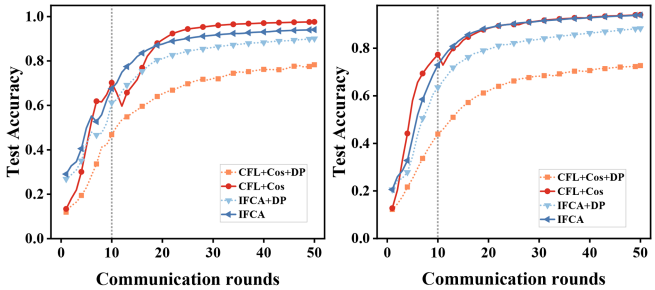
Fig. 6. Comparison of testing accuracy on Texas-100.

Furthermore, we observe from Fig. 3(b) that when $\lambda = 0.2$, ProCFL undergoes an accuracy degradation in the 10-th model training round, which has also been witnessed by other CFL schemes. Such an observation indicates that the global model obtained during the global learning phase is temporarily better than the cluster model obtained during the model personalization



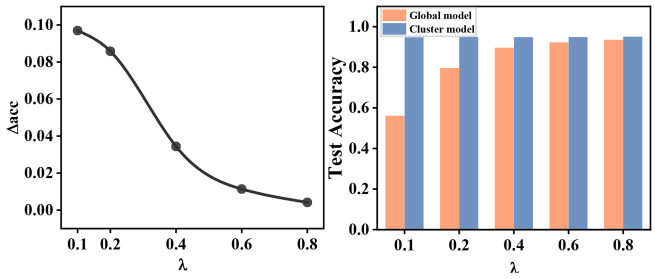
(a) The non-IID setting of $h = 20$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 7. Comparison of testing accuracy on TinyImage.



(a) The non-IID setting of $k = 2$ (b) The non-IID setting of $\beta = 0.5$

Fig. 8. Model test accuracy when applying DP to client clustering.

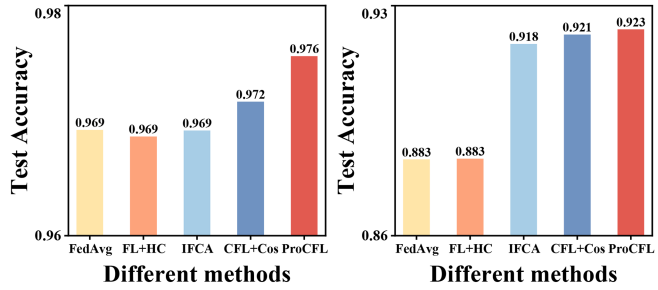


(a) Testing accuracy gap. (b) Testing accuracy of the global model and final cluster models.

Fig. 9. Impact of λ on ProCFL.

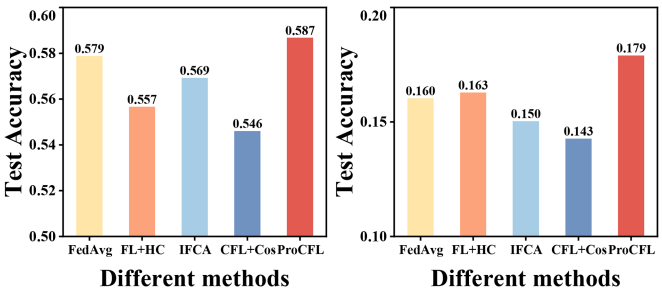
phase. Motivated by this, we conduct a series of experiments to explore the impact of λ on ProCFL following the experimental setting in Fig. 3(b), where we consider different values of λ including 0.1, 0.2, 0.4, 0.6, and 0.8, respectively. We present the accuracy gap (denoted as Δacc) between the global model in the $\lfloor \lambda T \rfloor$-th round and the cluster model in the $\lfloor \lambda T \rfloor + 1$-th round in Fig. 9(a). Intuitively, a smaller value of Δacc implies that the global model obtained in the first $\lfloor \lambda T \rfloor$ rounds is better utilized for personalized model training. Fig. 9(a) shows that a larger λ helps better utilize the global model. In addition, we also present the testing accuracies of the global model and final cluster models under different choices of λ in Fig. 9(b). We can see that the global model improves in λ, while the final cluster models can also reach a high accuracy level. In this way, both the server's and the clients' interests are guaranteed simultaneously.

We further evaluate the ProCFL under the non-IID setting of $\beta = 0.2$ to highlight our performance advantages under highly non-IID data scenarios. Here, we compare ProCFL and the



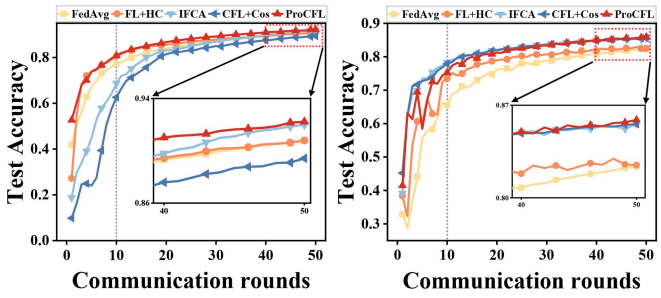
(a) Testing accuracy on MNIST. (b) Testing accuracy on FMNIST.

Fig. 10. Comparison of testing accuracy under non-IID setting of $\beta = 0.2$.



(a) Testing accuracy on Texas-100. (b) Testing accuracy on TinyImage.

Fig. 11. Comparison of testing accuracy under non-IID setting of $\beta = 0.2$.



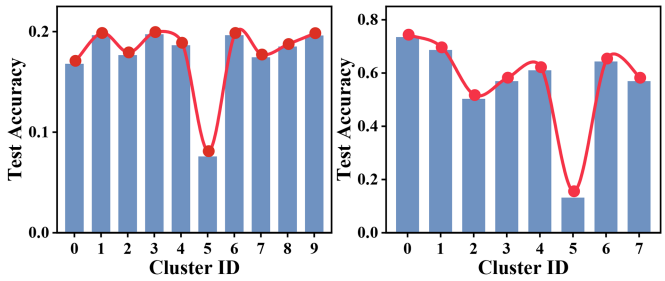
(a) Testing accuracy on MNIST. (b) Testing accuracy on FMNIST.

Fig. 12. Comparison of testing accuracy under feature skew setting.

baselines regarding the largest average accuracy of client models achieved throughout the model training process. Figs. 10 and 11 plots the results, validating that ProCFL consistently outperforms the baselines even with a higher non-IID degree. In addition, we evaluate the effectiveness of our scheme in the feature skew scenario. As demonstrated in Fig. 12, our scheme still possesses better performance than the baselines.

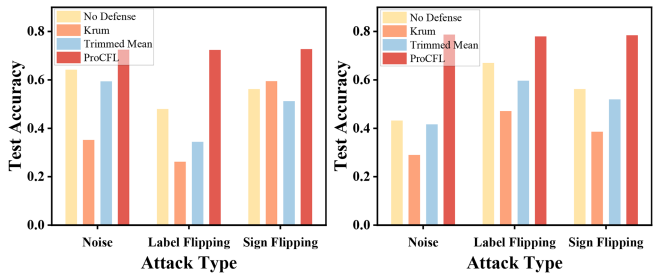
4) Malicious Cluster Attack Detection Performance: In the following, we evaluate the effectiveness of the proposed post-hoc malicious cluster attack detection mechanism. First, we evaluate scenario where malicious clients dominate a cluster. We then evaluate the scenario where malicious clients are scattered across multiple clusters. Finally, we evaluate the impact of different parameters on the detection mechanism.

a) Malicious cluster attack evaluation: We first consider that only one malicious cluster exists, where malicious clients are in the majority. To this end, after accomplishing client clustering, we randomly select one cluster as the malicious cluster and



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 13. Malicious cluster detection in non-IID settings.



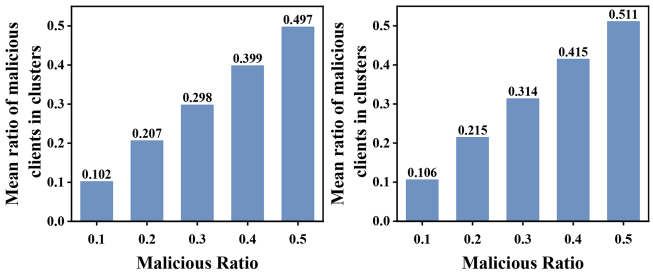
(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 14. Malicious cluster attack detection results under stronger attack.

assume that 60% of its clients are compromised. Each malicious client launches the Gaussian attack [54], label flipping attack [55] and sign flipping attack [56]. Here, the noise variance δ of Gaussian attack is set to 1, which can lead to significant degradation in the model accuracy. The detection parameter d of our detection mechanism is set to the size of the identified malicious cluster, i.e., the number of clients therein. We compare our detection mechanism with Krum [37] and Trimmed Mean [57].

In this experiment, we set the malicious clients to cluster in cluster 5, and set the server to detect the accuracy of all cluster models in the last round of the global learning phase (i.e., $t = \lfloor \lambda T \rfloor$). Fig. 13 shows the evaluation results of the server, and it is clear that the cluster model of the malicious cluster has a lower accuracy, and the server can assume that the corresponding cluster is suspected. Fig. 14 shows the effect of ProCFL excluding malicious model updates after finding malicious clusters. When the number of malicious clients in the cluster exceeds 50%, mainstream robust aggregation algorithms will fail, while our method is still able to exclude malicious clients.

b) Dispersed attack evaluation: Next, we consider a general case of malicious cluster attacks, where malicious clients are dispersed across multiple clusters instead of gathering in several specific clusters. The proportion of total malicious clients over total clients is varied from 0.1 to 0.5. Fig. 15 depicts the average ratio of malicious clients in each cluster, which increases with the overall ratio of malicious clients. Nevertheless, as long as the overall ratio of malicious clients remains below 50%, the average ratio of malicious clients in each cluster does not exceed 50% either. This finding indicates that it is difficult for malicious clients to dominate a cluster in this case.



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 15. The average ratio of malicious clients in each cluster.

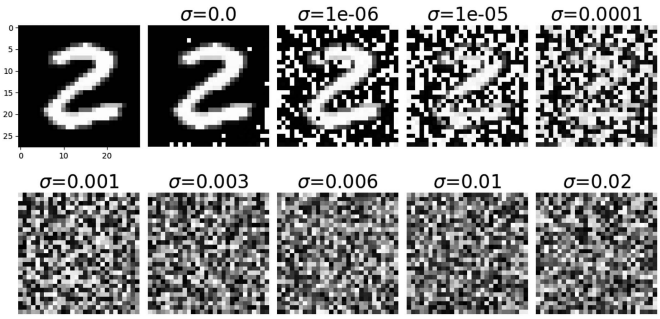
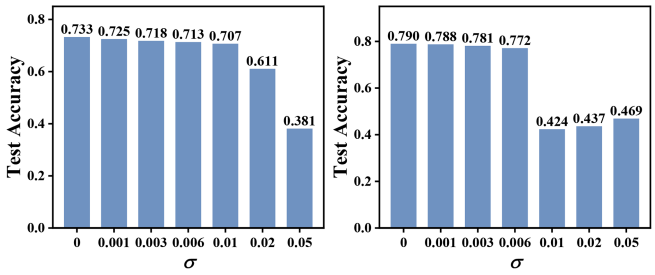
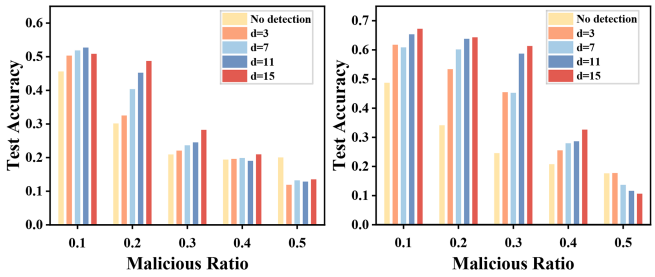


Fig. 16. DLG-based data reconstruction results under varying noise variances.



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 17. Trade-off between privacy (reflected by the noise variance) and the global model accuracy.



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 18. Test accuracy after detection with different d.

In the subsequent experiments, we consider that malicious clients dispersedly execute model poisoning attacks in the $\lfloor \lambda T \rfloor$-th round without loss of generality. To identify malicious cluster models, the server examines their testing accuracies and compares them with respective accuracies from the $(\lfloor \lambda T \rfloor - 1)$-th round. If a degradation in accuracy is detected for a given cluster model, it is flagged as malicious. Our post-hoc detection mechanism is then employed to identify the malicious local models within the cluster.

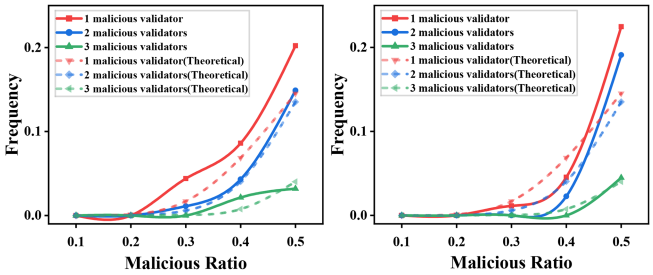
3) Trade-off between privacy and utility: To detect malicious local models, our mechanism requires clients to submit their raw local models to a validation committee for evaluation. However, this poses a privacy risk, as *honest-but-curious* committee members could infer sensitive information from the shared models. To mitigate this concern, we offer clients the option to obfuscate their models using the Gaussian mechanism [15] with a noise variance of σ.

We first examine the relationship between perturbation level (measured by Gaussian noise variance) and privacy protection. Specifically, we add varying levels of noise to the shared local models and apply the well-known privacy attack *Deep Leakage from Gradients* (DLG), which reconstructs training data from shared gradients [15]. Fig. 16 demonstrates that when the noise variance $\sigma \geq 0.001$, data reconstruction from gradients becomes infeasible, effectively preventing privacy leakage. Next, we assess the impact of perturbation level on utility, measured by the test accuracy of the final global model. Fig. 17 shows that, under typical non-IID data distributions (e.g., $h = 2$ in the quantity-based label imbalance setting and $\beta = 0.5$ in the distribution-based label imbalance setting), a noise variance $\sigma \leq 0.006$ allows the global model to achieve desirable accuracy. Based on these experiments, we conclude that, within the tested settings, a Gaussian noise variance between $0.001 \leq \sigma \leq 0.006$ ensures adequate privacy protection while maintaining desirable global model accuracy. We believe that similar perturbation levels can be identified for other experimental settings to balance privacy and utility.

4) Impact of d on Detection Performance: In addition, since malicious clients are dispersed across multiple clusters, there is a possibility that some malicious clients may be selected for the validation committee. Suppose that malicious clients in the committee determine whether a client model in the malicious cluster is malicious in an opposite way to that used in (14). Fig. 18 shows our detection performance (as indicated by the global model accuracy) for different values of d. The results indicate that our mechanism achieves satisfactory detection performance when the overall ratio of malicious clients is less than 50%. Furthermore, a larger value of d, which translates to a greater number of members in the validation committee, generally leads to better detection performance.

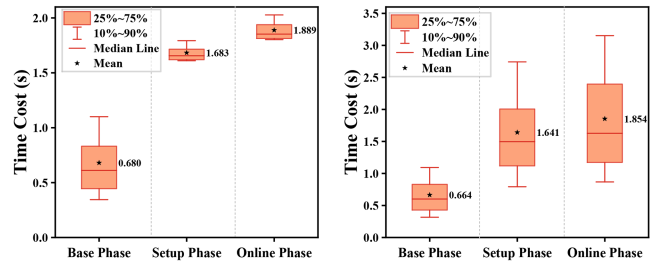
5) Evaluation on Credibility of Validation Committee: Finally, we examine the frequency of the validation committee containing 1, 2, and 3 malicious clients when malicious cluster attacks occur under $d = 3$. Fig. 19 displays the results, including theoretical values from Section V for reference. The findings demonstrate that malicious clients have a low probability of appearing in the validation committee, which aligns with our theoretical analysis.

5) Computation Overhead: We first investigate the computation overhead (captured by time cost) introduced by the data distribution similarity measurement process (i.e., the RSA-PSI [44] based ISM vectors calculation).



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 19. Frequencies of malicious clients appearing in the committee.



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 20. Time cost for privacy-preserving ISM calculation.

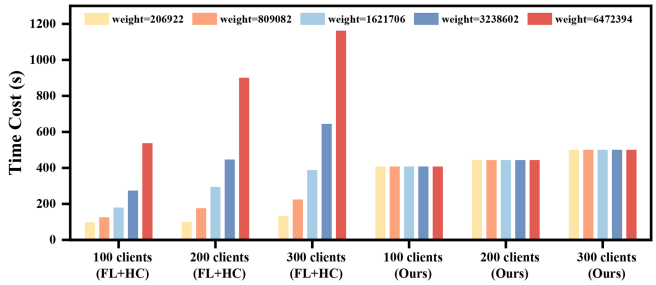
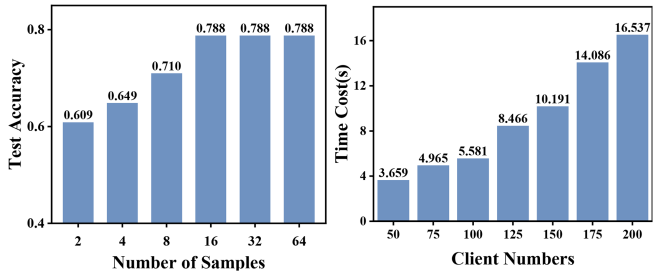


Fig. 21. Time cost for clustering operation.



(a) Test accuracy of the global model under different numbers of local data samples used for testing.

(b) Time cost of detecting malicious clients.

Fig. 22. Computational overhead of the peer-validation-based malicious client detection mechanism.

Specifically, we consider $N = 100$ clients, and each of them computes its ISM vector *w.r.t* others under the non-IID settings of $h = 2$ and $\beta = 0.5$ using the RSA-PSI [44] scheme that consists of three phases (i.e., base phase, setup phase, and online phase). In particular, we record the time cost incurred by each client in each phase of the RSA-PSI-based ISM vector calculation.

Fig. 20 illustrates that during each phase of the RSA-PSI-based ISM vector calculation, approximately 25% to 75% of all clients incur a time cost centered around the mean value of that particular phase. Specifically, for the non-IID setting with $h = 2$ as shown in Fig. 20(a), the average time cost for each phase is 0.680s, 1.683s, and 1.889s, respectively. Similarly, for the non-IID setting with $\beta = 0.5$ as depicted in Fig. 20(b), the average time cost is 0.644s, 1.641s, and 1.854s, respectively. Evidently, the time cost associated with our proposed RSA-PSI-based ISM vector calculation is almost negligible compared to the time expended on model training. Furthermore, the process of measuring data distribution similarity can be performed offline, with the results stored prior to initiating the model training task.

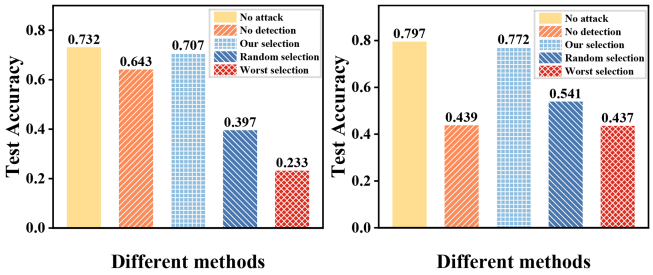
To further investigate the computation efficiency of our clustering method, we compare with the CFL scheme that incurs minimal overhead, specifically the one-shot clustering scheme FL+HC [14]. Fig. 21 demonstrates that FL+HC, which relies on gradients for clustering, suffers from a significant increase in overhead as the number of model weights and clients grows. In contrast, our privacy-preserving clustering scheme, which does not rely on gradients, scales well in model size and the number of clients.

To evaluate the computational overhead incurred by validation committee members when assessing local model performance, we conducted experiments to analyze the impact of the number of local data samples used for testing on malicious client detection performance, as measured by global model accuracy. The results, presented in Fig. 22(a), demonstrate that using only 16 local data samples for model performance assessment ensures desirable global model accuracy. Importantly, the computational overhead for validation committee members in this process (involving only forward propagation) is significantly lower than that required for their participation in CFL, which involves local model training (involving both forward and backpropagation). Fig. 22(b) shows that the increase in the time cost of detecting malicious clients is roughly linear with the number of clients, which verifies that the time complexity of our malicious client detection mechanism is acceptable.

6) Committee Member Selection Performance: In this subsection, we evaluate the effectiveness of our proposed validation committee member selection mechanism (i.e., selecting those clients with the highest ISM values *w.r.t.* the clients in the malicious cluster).

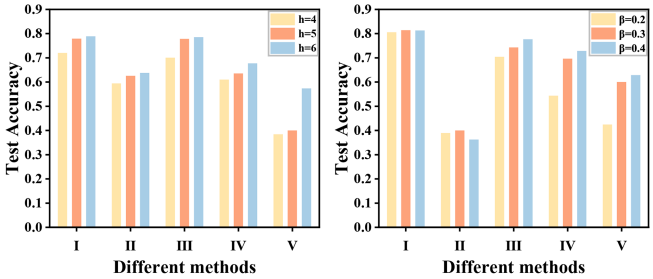
We first compare our mechanism with two baselines, i.e., random selection and the mechanism that selects those clients with the lowest ISM values *w.r.t.* the clients in the malicious cluster (called worst selection). In the following experiments, inspired by Fig. 17, we respectively choose $\sigma = 0.01$ and $\sigma = 0.006$ for the non-IID settings of $h = 2$ and $\beta = 0.5$ to provide sufficient privacy protection for clients while maintaining desirable detection performance.

Fig. 23 shows that our designed committee member selection mechanism significantly outperforms the two baselines (i.e., yielding a higher global model accuracy), with performance comparable to the case where no attacks happen. Conversely,



(a) The non-IID setting of $h = 2$. (b) The non-IID setting of $\beta = 0.5$.

Fig. 23. Test accuracy with different validation committee member selection mechanisms.



(a) Different non-IID degrees of $h = 4, 5, 6$. (b) Different non-IID degrees of $\beta = 0.2, 0.3, 0.4$.

Fig. 24. Test accuracy with different validation committee member selection mechanisms (I: No attack, II: No detection, III: Our selection, IV: Random selection, V: Worst selection) under different non-IID degrees.

under the non-IID setting of $h = 2$, the global model accuracy achieved via random selection is inferior to that trained without performing any malicious model detection because randomly selected validation committee members may not have similar data distributions with the clients in the malicious cluster, leading to inaccurate predictions about whether a client is malicious or not. Besides, the selection mechanism that uses the opposite criteria to the proposed method performs poorly in detecting malicious clients.

We further investigate the performance of our selection mechanism under different non-IID degrees. The results, depicted in Fig. 24, indicate that as the non-IID degree increases (with smaller values of h and β), there is a slight decline in the performance of our mechanism, as evidenced by the testing accuracy of the global model. However, it is important to note that our mechanism maintains a significant advantage over other methods. Furthermore, our mechanism demonstrates comparable performance compared to the non-attack scenario.

VII. Discussions

Limitations: Even though the ISM calculation can be conducted offline before model training commences, it does come with additional time costs. We leverage the PSI scheme in [44] as it enables clients to dynamically add training samples with minimal overhead. However, designing a more efficient PSI protocol to lower computation overhead is outside the scope of this work. Additionally, while ProCFL can effectively combat malicious cluster attacks, this study does not address detecting and removing malicious clients (independent of malicious local models).

Extensions: This work focuses on achieving efficient client clustering in CFL without relying on clients' gradient information or other prior knowledge, which enhances privacy protection. This is accomplished by designing a gradient-free data distribution similarity metric (i.e., ISM). However, we do not address the design of privacy preservation techniques to safeguard against gradient leakage attacks during the FL model training process, as several popular schemes, such as differential privacy [58], [59] and secure aggregation [16], can handle these privacy issues. Our proposed diversity-optimized clustering algorithm is independent of these privacy protection techniques as it does not depend on gradients. Therefore, adding perturbation noises to gradients or performing cryptographic operations during the model training process does not compromise the effectiveness of our scheme.

More Attacks: Malicious clients may attempt poisoning attacks during the model personalization phase. Our malicious client detection mechanism is also applicable to such attacks. Specifically, in our design, clients with diverse data classes are clustered into multiple groups. During each round of model personalization, a client receives multiple cluster models from different cluster leaders and evaluates these models using its local training data. If a cluster model is identified as potentially poisoned by a malicious client, the client can notify the corresponding cluster leader to initiate the peer-validation process. This involves requesting the suspected client to share their local models with the validation committee members for further detection.

VIII. Conclusions and Future Work

This paper presented the ProCFL framework, which achieves efficient and robust model training. We propose the ISM data distribution similarity metric to enable client clustering without compromising client privacy. Furthermore, we transform the client clustering process into a WSCP and design a greedy diversity-optimized clustering algorithm for exploiting data diversity to optimize model performance. Furthermore, we develop a post-hoc malicious cluster attack detection mechanism to detect and remove malicious local models after identifying malicious cluster models. Experimental evaluations demonstrate that ProCFL offers superior performance compared to existing solutions, delivering high model accuracy and robustness while preserving client privacy during the client clustering process.

In this work, we mainly focus on two common non-IID settings. Despite the promising results of ProCFL, it remains a very challenging problem to design more general metrics to measure the similarity of data distribution under privacy protection for various non-IID data scenarios. In the future, we will consider designing more general data distribution similarity metrics while prioritizing privacy protection. We will also concentrate on more practical non-IID scenarios, such as federated recommendation systems, to enhance their effectiveness in real-world applications.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[2] B. Luo, P. Han, P. Sun, X. Ouyang, J. Huang, and N. Ding, "Optimization design for federated learning in heterogeneous 6G networks," *IEEE Netw.*, vol. 37, no. 2, pp. 38–43, Mar./Apr. 2023.

[3] X. Pang, J. Hu, P. Sun, J. Ren, and Z. Wang, "When federated learning meets knowledge distillation," *IEEE Wireless Commun.*, vol. 31, no. 5, pp. 208–214, Oct. 2024.

[4] Y. Zhou, X. Pang, Z. Wang, J. Hu, P. Sun, and K. Ren, "Towards efficient asynchronous federated learning in heterogeneous edge environments," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2024, pp. 2448–2457.

[5] R. Lu et al., "Auction-based cluster federated learning in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1145–1158, Apr. 2023.

[6] F. Sattler, K. Müller, and W. Samek, "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 8, pp. 3710–3722, Aug. 2021.

[7] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," *IEEE Trans. Inf. Theory*, vol. 68, no. 12, pp. 8076–8091, Dec. 2022.

[8] Z. He, L. Wang, and Z. Cai, "Clustered federated learning with adaptive local differential privacy on heterogeneous IoT data," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 137–146, Jan. 2024.

[9] S. D. Okegbile, J. Cai, H. Zheng, J. Chen, and C. Yi, "Differentially private federated multi-task learning framework for enhancing human-to-virtual connectivity in human digital twin," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3533–3547, Nov. 2023.

[10] K. Wei et al., "Personalized federated learning with differential privacy and convergence guarantee," *IEEE Trans. Inf. Forensic Secur.*, vol. 18, pp. 4488–4503, 2023.

[11] R. Mishra, H. P. Gupta, G. Banga, and S. K. Das, "Fed-RAC: Resource aware clustering for tackling the heterogeneity of participants in federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 7, pp. 1207–1220, Jul. 2024.

[12] R. Zhang et al., "CPPer-FL: Clustered parallel training for efficient personalized federated learning," *IEEE. Trans. Mobile. Comput.*, vol. 23, no. 10, pp. 9424–9436, Oct. 2024.

[13] Y. Mansour, M. Mohri, J. Ro, and A. Suresh, "Three approaches for personalization with applications to federated learning," 2020, *arXiv: 2002.10619*.

[14] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-IID data," in *Proc. Int. J. Conf. Neural Netw.*, 2020, pp. 1–9.

[15] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Proces. Syst.*, 2019, pp. 14774–14784.

[16] Y. Li, J. Lai, R. Zhang, and M. Sun, "Secure and efficient multi-key aggregation for federated learning," *Inf. Sci.*, vol. 654, pp. 119830–119849, 2024.

[17] P. Sun, X. Chen, G. Liao, and J. Huang, "A profit-maximizing model marketplace with differentially private federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1439–1448.

[18] P. Sun, G. Liao, X. Chen, and J. Huang, "A socially optimal data marketplace with differentially private federated learning," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2221–2236, Jun. 2024.

[19] L. Cai, N. Chen, Y. Cao, J. He, and Y. Li, "FedCE: Personalized federated learning method based on clustering ensembles," in *Proc. ACM Int. Conf. Multimedia*, 2023, pp. 1625–1633.

[20] S. AbdulRahman, S. Otoum, O. Bouachir, and A. Mourad, "Management of digital twin-driven IoT using federated learning," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3636–3649, Nov. 2023.

[21] X. Li, Z. Qu, S. Zhao, B. Tang, Z. Lu, and Y. Liu, "LoMar: A local defense against poisoning attack on federated learning," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 1, pp. 437–450, Jan. 2023.

[22] C. Ma, J. Li, M. Ding, K. Wei, W. Chen, and H. V. Poor, "Federated learning with unreliable clients: Performance analysis and mechanism design," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17308–17319, Dec. 2021.

[23] Z. Li, L. Liu, J. Zhang, and J. Liu, "Byzantine-robust federated learning through spatial-temporal analysis of local model updates," in *Proc. Int. Conf. Parallel Distrib. Syst.*, 2021, pp. 372–379.

[24] L. Zhao et al., "Shielding collaborative learning: Mitigating poisoning attacks through client-side detection," *IEEE Trans. Dependable Secur. Comput.*, vol. 18, no. 5, pp. 2029–2041, May 2021.

[25] P. Sun, X. Liu, Z. Wang, and B. Liu, "Byzantine-robust decentralized federated learning via dual-domain clustering and trust bootstrapping," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 24756–24765.

[26] B. Zhao, P. Sun, T. Wang, and K. Jiang, "Fedinv: Byzantine-robust federated learning by inversing local model updates," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9171–9179.

[27] Z. Li, Z. Guan, S. Yuan, N. An, and X. Liang, "ROCFL: A robust clustered federated learning framework towards heterogeneous data," in *Proc. Int. Conf. Intell. Commun. Netw.*, 2023, pp. 259–264.

[28] J. Ma, M. Xie, and G. Long, "Personalized federated learning with robust clustering against model poisoning," in *Proc. Adv. Data Mining Appl.*, 2022, pp. 238–252.

[29] Y. Li, D. Yuan, A. S. Sani, and W. Bao, "Enhancing federated learning robustness in adversarial environment through clustering non-IID features," *Comput. Secur.*, vol. 132, pp. 103319–103332, 2023.

[30] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE. Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3805–3822, Jul. 2023.

[31] Y. Diao, Q. Li, and B. He, "Exploiting label skews in federated learning with model concatenation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 11784–11792.

[32] C. Fan, R. Chen, J. Mo, and L. Liao, "Personalized federated learning for cross-building energy knowledge sharing: Cost-effective strategies and model architectures," *Appl. Energy*, vol. 362, pp. 123016–123031, 2024.

[33] Z. Ye, W. Luo, Q. Zhou, and Y. Tang, "High-fidelity gradient inversion in distributed learning," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 19983–19991.

[34] F. Wang, E. Hugh, and B. Li, "More than enough is too much: Adaptive defenses against gradient leakage in production federated learning," *IEEE-ACM Trans. Netw.*, vol. 32, pp. 3061–3075, 2024.

[35] J. Wu, M. Hayat, M. Zhou, and M. Harandi, "Concealing sensitive samples against gradient leakage in federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 21717–21725.

[36] Y. Ruan and C. Joe Wong, "FedSoft: Soft clustered federated learning with proximal local updating," in *Proc. AAAI Conf. Artif. Intell.*2022, pp. 8124–8131.

[37] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Proces. Syst.*, 2017, pp. 118–128.

[38] L. Zhao, J. Jiang, B. Feng, Q. Wang, C. Shen, and Q. Li, "SEAR: Secure and efficient aggregation for Byzantine-robust federated learnin," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 5, pp. 3329–3342, Sep./Oct. 2022.

[39] C. Zhang et al., "A blockchain-based model migration approach for secure and sustainable federated learning in IoT systems," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6574–6585, Aug. 2023.

[40] Q. Li, Z. Liu, Q. Li, and K. Xu, "martFL: Enabling utility-driven data marketplace with a robust and verifiable federated learning architecture," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1496–1510.

[41] Y. Lu et al., "Federated learning with extremely noisy clients via negative distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 14184–14192.

[42] L. Xie et al., "CNTSeg: A multimodal deep-learning-based network for cranial nerves tract segmentation," *Med. Image Anal.*, vol. 86, pp. 102766–102781, 2023.

[43] D. Morales, I. Agudo, and J. Lopez, "Private set intersection: A systematic literature review," *Comput. Sci. Rev.*, vol. 49, pp. 100567–100591, 2023.

[44] Á. Kiss, J. Liu, T. Schneider, N. Asokan, and B. Pinkas, "Private set intersection for unequal set sizes with mobile applications," *Privacy Enhancing Technol.*, vol. 2017, no. 4, pp. 177–197, 2017.

[45] J. Yang and J. Leung, "A generalization of the weighted set covering problem," *Nav. Res. Logistics*, vol. 52, no. 2, pp. 142–149, 2005.

[46] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Jun. 2012.

[47] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv: 1708.07747*.

[48] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," 2009.

[49] Y. Le and X. Yang, "Tiny imagenet visual recognition challenge," *CS 231 N*, vol. 7, no. 7, 2015, Art. no. 3.

[50] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 3–18.
[51] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 965–978.
[52] M. Nasr, R. Shokri, and A. Houmansadr, "Machine learning with membership privacy using adversarial regularization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 634–646.
[53] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[54] C. Ma et al., "On safeguarding privacy and security in the framework of federated learning," *IEEE Netw.*, vol. 34, no. 4, pp. 242–248, 2020.
[55] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. USENIX Secur. Symp.*, 2020, pp. 1605–1622.
[56] S. P. Karimireddy, L. He, and M. Jaggi, "Learning from history for Byzantine robust optimization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5311–5319.
[57] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.
[58] G. Huang, Q. Wu, P. Sun, Q. Ma, and X. Chen, "Collaboration in federated learning with differential privacy: A stackelberg game analysis," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 455–469, Mar. 2024.
[59] P. Sun, L. Wu, Z. Wang, J. Liu, J. Luo, and W. Jin, "A profit-maximizing data marketplace with differentially private federated learning under price competition," in *Proc. ACM Manage. Data*, vol. 2, no. 4, pp. 1–27, 2024.

Yang Xu received the PhD degree in computer science and technology from Central South University, China, in 2019. From 2015 to 2017, he was a visiting scholar in the Department of Computer Science and Engineering with Texas A&M University, USA. He is currently an associate professor with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include distributed computing, trustworthy computing, blockchain, and federated learning. He has published more than 50 articles in international journals and conferences, including *IEEE Transactions on Services Computing*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Cloud Computing*, etc. He was the awardee of the Best Paper Award of IEEE IoP 2018. He has served as the General co-chair for C4W 2022, the Steering Committee co-chair for IWCSS 2022, the TPC co-chair for UbiSec 2021, and an active reviewer for more than 20 international journal/conference proceedings. He is a member of the Blockchain Technical Committee of China Computer Federation (CCF) and China Society for Industrial and Applied Mathematics (CSIAM), a senior member of CCF.

Yunlin Tan received the BS degree in communication engineering from Hunan Normal University, Hunan, China, in 2021. He is currently working toward the MS degree in information and communication engineering with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include blockchain and federated learning.

Cheng Zhang received the BS degree in computer science and technology from the Shenyang University of Technology, China, in 2017, and the MS degree from Central South University, China, in 2021. He is currently working toward the PhD degree with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include network security and blockchain.

Peng Sun received the BE degree in automation from Tianjin University, China, in 2015, and the PhD degree in control science and engineering from Zhejiang University, China, in 2020. From Dec. 2020 to Oct. 2022, he worked as a postdoctoral researcher with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen. He is currently an associate professor with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include the Internet of Things, mobile crowdsensing, and federated learning.

Yibang Zhang received the BS degree in Internet of Things engineering from the Hunan Normal University, China, in 2022. He is currently working toward the MS degree with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include federated learning and blockchain.

Ju Ren (Senior Member, IEEE) received the BS, MS, and PhD degrees all in computer science, from Central South University, China, in 2009, 2012, and 2016, respectively. Currently, he is an associate professor with the Department of Computer Science and Technology, Tsinghua University, China. His research interests include Internet-of-Things, edge computing, operating systems, as well as security and privacy. He currently serves as an associate editor for *IEEE Transactions on Vehicular Technology and Peer-to-Peer Networking and Applications*. He also served as the general co-chair for IEEE BigDataSE'20, the TPC co-chair for IEEE BigDataSE'19, a poster co-chair for IEEE MASS'18, a track co-chair for IEEE/CIC ICCC'19, IEEE I-SPAN'18 and VTC'17 Fall, etc. He received many best paper awards from IEEE flagship conferences, including IEEE ICC'19 and IEEE HPCC'19, etc., the IEEE TCSC Early Career Researcher Award (2019), and the IEEE ComSoc Asia-Pacific Best Young Researcher Award, in 2021. He was recognized as a highly cited Researcher by Clarivate (2020-2022).

Hongbo Jiang (Senior Member, IEEE) received the PhD degree from Case Western Reserve University, in 2008. He was a professor with the Huazhong University of Science and Technology. He is currently a full professor with the College of Computer Science and Electronic Engineering, Hunan University. His research interests include computer networking, especially algorithms, and protocols for wireless and mobile networks. He is an Elected Member of Academia Europaea, a fellow of IET, a fellow of BCS, and a fellow of AAIA. He was the editor of *IEEE/ACM Transactions on Networking*, the associate editor of *IEEE Transactions on Mobile Computing*, and the associate technical editor of *IEEE Communications Magazine*.

Yaoxue Zhang (Senior Member, IEEE) received the BS degree from the Northwest Institute of Telecommunication Engineering, China, in 1982, and the PhD degree in computer networking from Tohoku University, Japan, in 1989. He is currently a professor with the Department of Computer Science and Technology, Tsinghua University, Beijing, China. His research interests include computer networking, operating systems, and transparent computing. He has published over 200 papers. He is the editor-in-chief of the chinese *Journal of Electronics* and a fellow of the *Chinese Academy of Engineering*.